

Workshop 2

Design Artifacts and System Modeling

Students:

Juan David Castaneda Cardenas

Laura Vanesa Alvarez Lafont

Nicolas Andres Bolanos Fernandez

Santiago Andres Fetecua Pulgarin

Sergio Nicolas Correa Escobar

Professor:

Carlos Andres Sierra Virquez

**Software Engineering II
School of Engineering
Universidad Nacional de Colombia**

1. CRC Cards

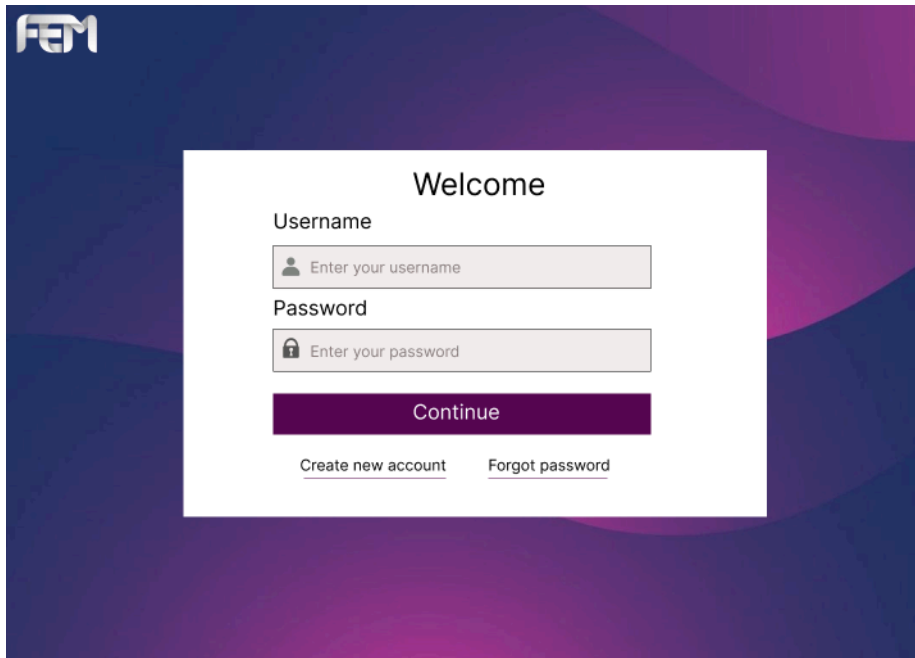
Organizer User	
Responsibilities	Collaboration
Knows identity information: username, email, password Authenticates with the system Owns zero or more Events (association) Can modify the list of owned Events -creating new events or deleting them-	Event Event

Event	
Responsibilities	Collaboration
Knows name Knows date Knows time Knows location Knows capacity Maintains list of attendees Computes number of confirmed attendees Belongs to a Organizer User	Attendance Attendance Organizer User

Attendance	
Responsibilities	Collaboration
Knows attendee name Knows attendee email Knows attendee contact number Knows attendee document ID Stores attendance status (confirmed / not confirmed) Stores waitlist status (admitted / waiting) Belongs to an Event	Attendance Attendance Event

These cards show the principal classes of the project, their responsibilities and the way in which they collaborate.

2. Mock Up



The login screen features a dark purple and blue wavy background. In the top left corner is the 'FEM' logo. A white rectangular card is centered on the screen. At the top of the card is the heading 'Welcome'. Below it are two input fields: 'Username' with a person icon and 'Password' with a lock icon. A purple 'Continue' button is positioned below the password field. At the bottom of the card are two links: 'Create new account' and 'Forgot password'.

Welcome

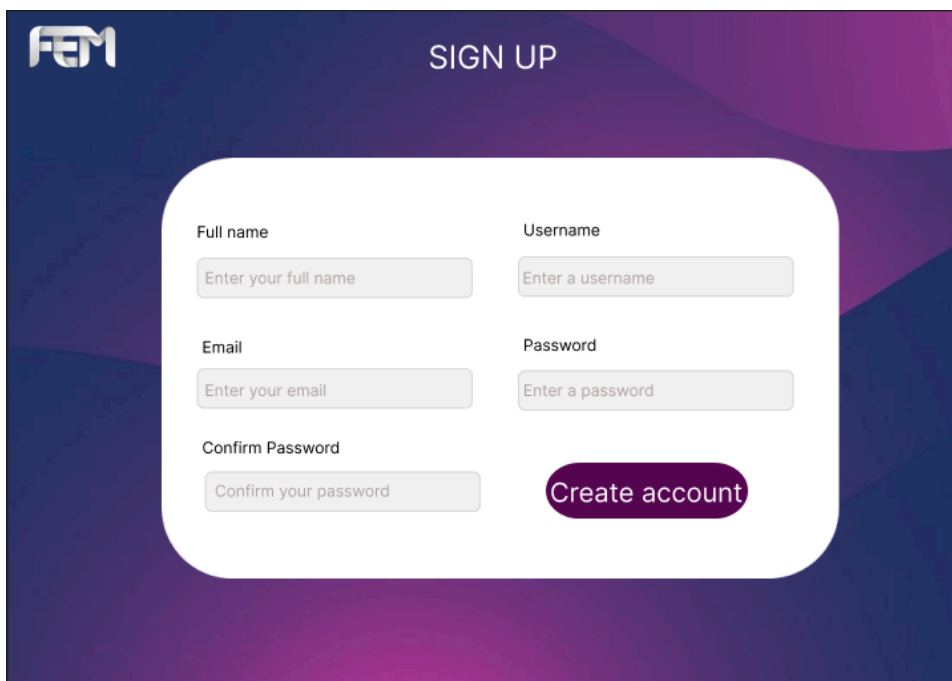
Username
Enter your username

Password
Enter your password

Continue

[Create new account](#) [Forgot password](#)

The Login Screen is the entry point of the application and is used by the organizer user to sign into their account. When the user enters their information and clicks Continue, the system verifies the credentials.



The sign up screen has the same dark purple and blue wavy background and 'FEM' logo as the login screen. A white rounded rectangular card is centered. At the top of the card is the heading 'SIGN UP'. Below it are four input fields arranged in two columns: 'Full name', 'Email', 'Username', and 'Password'. A 'Confirm Password' field is located below the 'Email' field. A purple 'Create account' button is positioned to the right of the 'Confirm Password' field.

SIGN UP

Full name
Enter your full name

Username
Enter a username

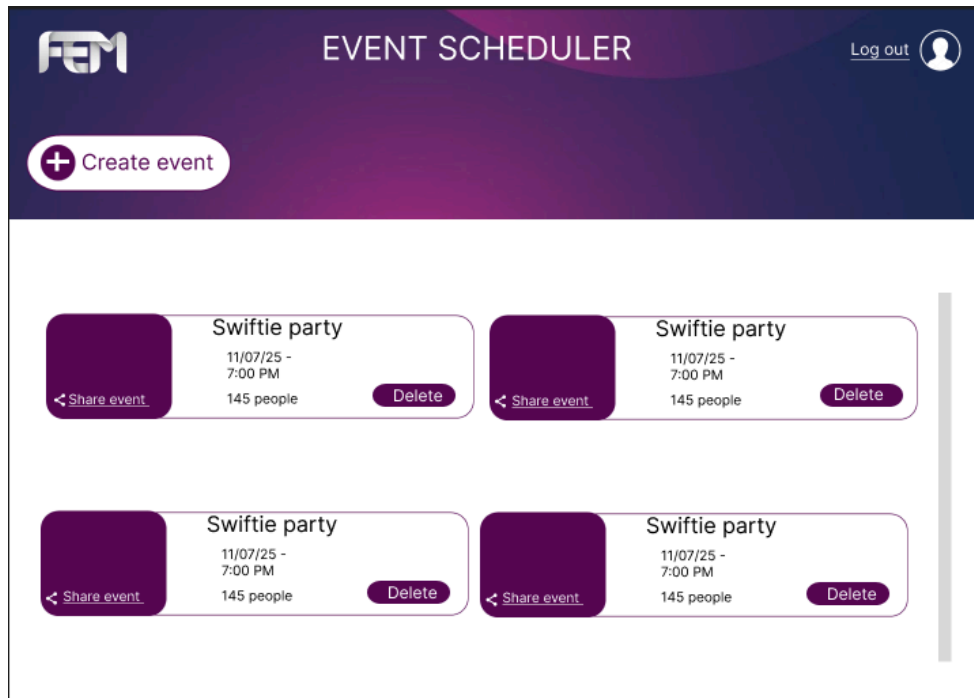
Email
Enter your email

Password
Enter a password

Confirm Password
Confirm your password

Create account

The Sign Up Screen is used by new organizer users to create an account in the system before accessing the event management features. This screen contains a form with fields to enter full name, username, email, password, and password confirmation, along with a “Create account” button that submits the information to the system. If everything is correct, a new account is registered and the user can then proceed to log in.



The Event Scheduler Screen is the main dashboard displayed to the organizer user after successfully logging into the system. Its purpose is to allow the organizer to view, manage, and interact with all the events they have created. The screen contains a “Create event” button at the top for adding new events, a list of event cards that show the event title, date and time, and the number of confirmed attendees, along with two actions for each event: “Share event” to copy or distribute the attendance link, and “Delete” to remove the event from the system. It also includes a “Log out” option at the top right to exit the session.

FEM **CREATE EVENT** [Back](#) [Log out](#) 

Event name

Location

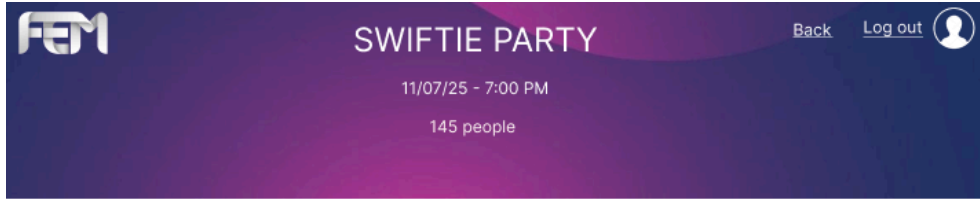
Date

Time

Maximum Capacity

Save

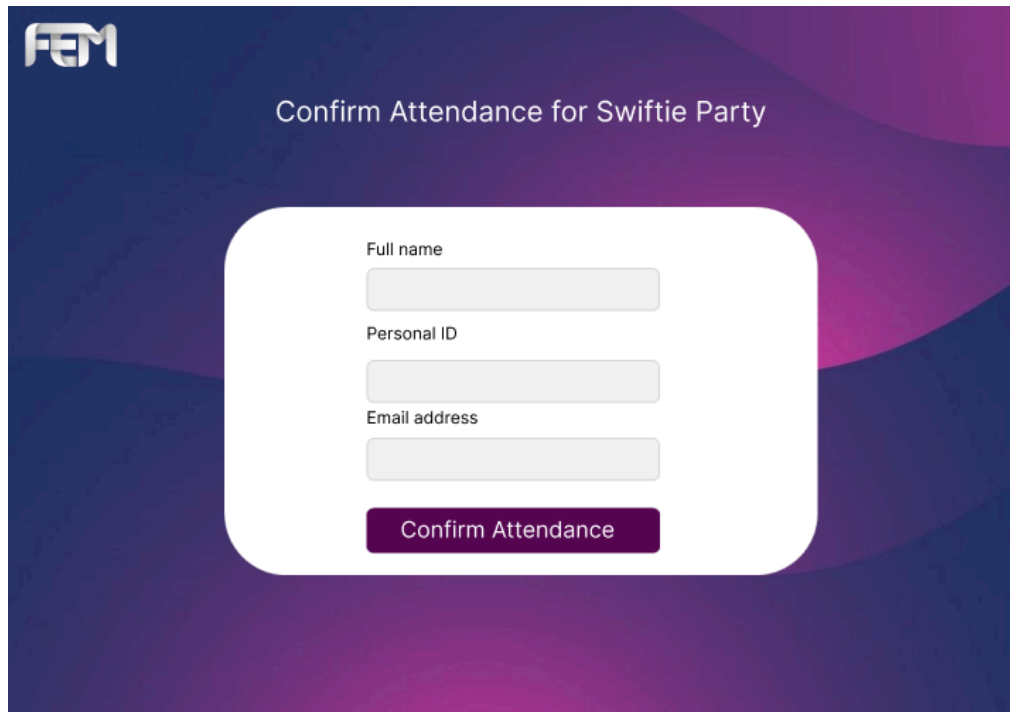
The Create Event Screen is used by the organizer user to register a new event in the system. This screen displays a form where the organizer can enter the event name, location, date, time, and maximum capacity, along with a “Save” button to confirm the creation of the event. When the user fills in all the required fields and clicks the Save button, the system validates the information and stores the new event in the database so that it appears later on the Event Scheduler dashboard. The screen also includes navigation options such as “Back” to return to the previous page and “Log out” to end the current session. This screen is only shown to organizer users who want to create new events.



Confirmed Guests List

Name	Email
Laura Ojeda	Ojedalaura12@gmail.com
Mariana Perez	Marianitapeq@gmail.com
Laura Ojeda	Ojedalaura12@gmail.com
Mariana Perez	Marianitapeq@gmail.com

The Event Details Screen is displayed to the organizer user when they select one of their created events from the main dashboard. Its purpose is to show the detailed information of the chosen event, including the event name, date, time, and the total number of confirmed attendees. Below this information, the screen presents a table titled “Confirmed Guests List,” which displays the names and contact emails of all guests who have confirmed their attendance. This allows the organizer to easily review, reference, or copy guest information if necessary. The screen also includes navigation elements such as a “Back” link to return to the event dashboard and a “Log out” option to close the session. This screen is used solely by the organizer to monitor and manage guest information for a specific event.



The image shows a web form titled "Confirm Attendance for Swiftie Party" with the FEM logo in the top left. The form is centered on a white background with rounded corners. It contains three input fields: "Full name", "Personal ID", and "Email address". Below these fields is a purple button labeled "Confirm Attendance". The background of the page is a dark blue and purple gradient with wavy patterns.

FEM

Confirm Attendance for Swiftie Party

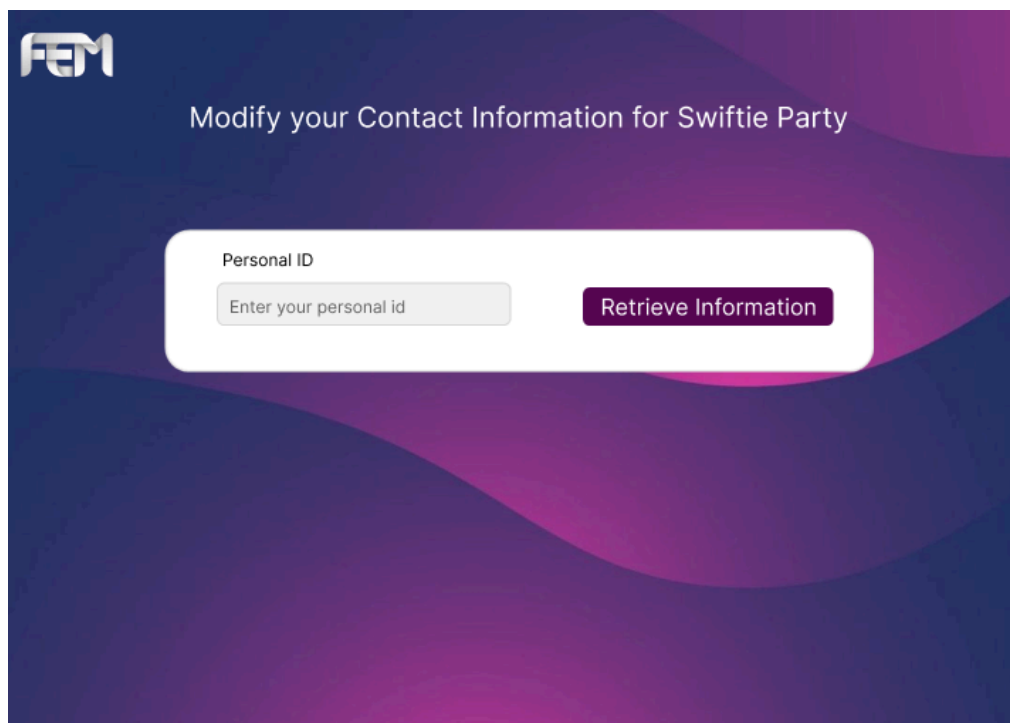
Full name

Personal ID

Email address

Confirm Attendance

Is designed for guest users who have received a link to confirm their participation in an event, in this case, the Swiftie Party. The interface presents a simple attendance confirmation form, which includes three input fields: Full name, Personal ID, and Email address. After filling in the information, users click the “Confirm Attendance” button to submit their details.



The image shows a web form titled "Modify your Contact Information for Swiftie Party" with the FEM logo in the top left. The form is centered on a white background with rounded corners. It contains one input field labeled "Personal ID" with the placeholder text "Enter your personal id". To the right of the input field is a purple button labeled "Retrieve Information". The background of the page is a dark blue and purple gradient with wavy patterns.

FEM

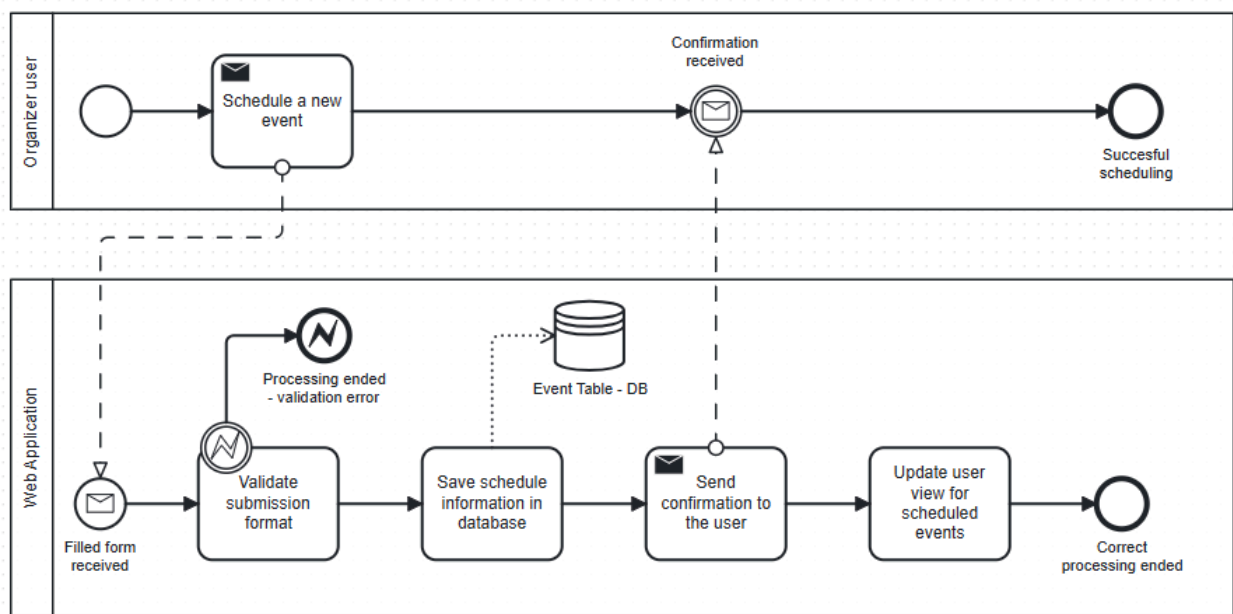
Modify your Contact Information for Swiftie Party

Personal ID

Retrieve Information

Allows a guest to modify their contact information for a specific event. The interface contains a single input field labeled Personal ID, where the user must enter their personal identification number. By clicking the “Retrieve Information” button, the system retrieves the guest’s previously stored data from the database, allowing them to update their personal details if needed. This functionality is intended for event guests who have already confirmed attendance, ensuring that the organizer has accurate and up-to-date contact information.

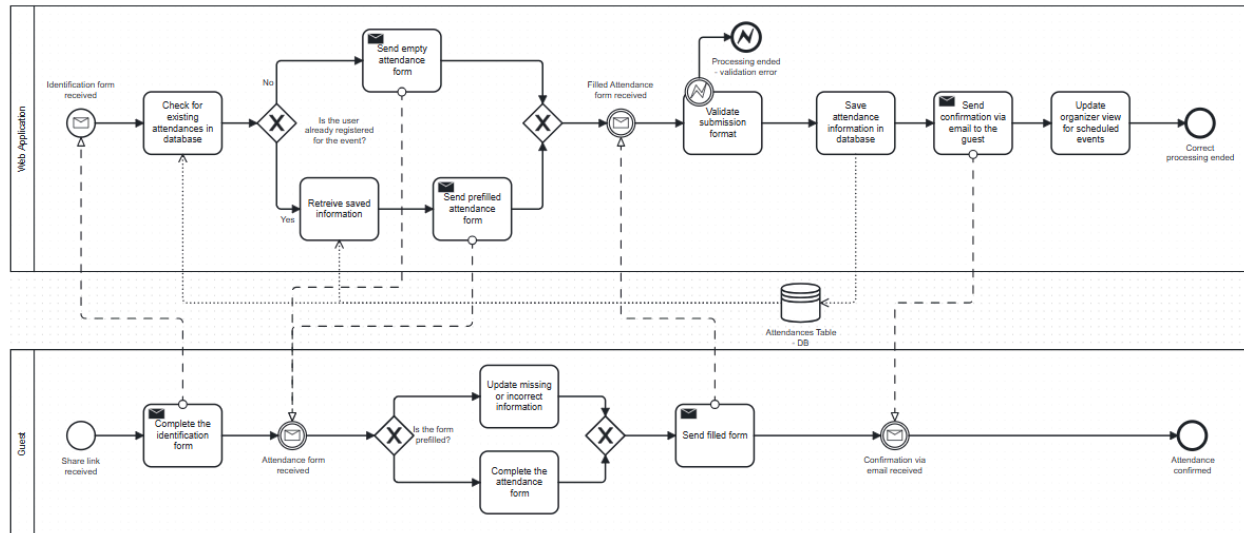
3. Business Model Processes



Process 1. Scheduling a new event (by an organizer)

Description: This process represents one of the core functionality of the application: an organizer user schedules a new event through the platform. It begins when the user decides to create a new event and completes the scheduling form. Then, the application receives this form and validates the format of the answers – if there is a format inconsistency, the system throws an error and the process ends–. After that, it saves the information into the system database, sends the user a confirmation message and updates the scheduled events of the user in the homepage.

Role in the application: This is the basis of functionality in the system, as it enables registered users to create and manage one of the main entities that the application is built around: the events. It ensures that new events are properly registered and persistent over time, in order to be visible in the user’s account. Other key activities (such as confirming attendance) depend on the correct execution of this process.



Process 2. Confirming attendance (by a guest)

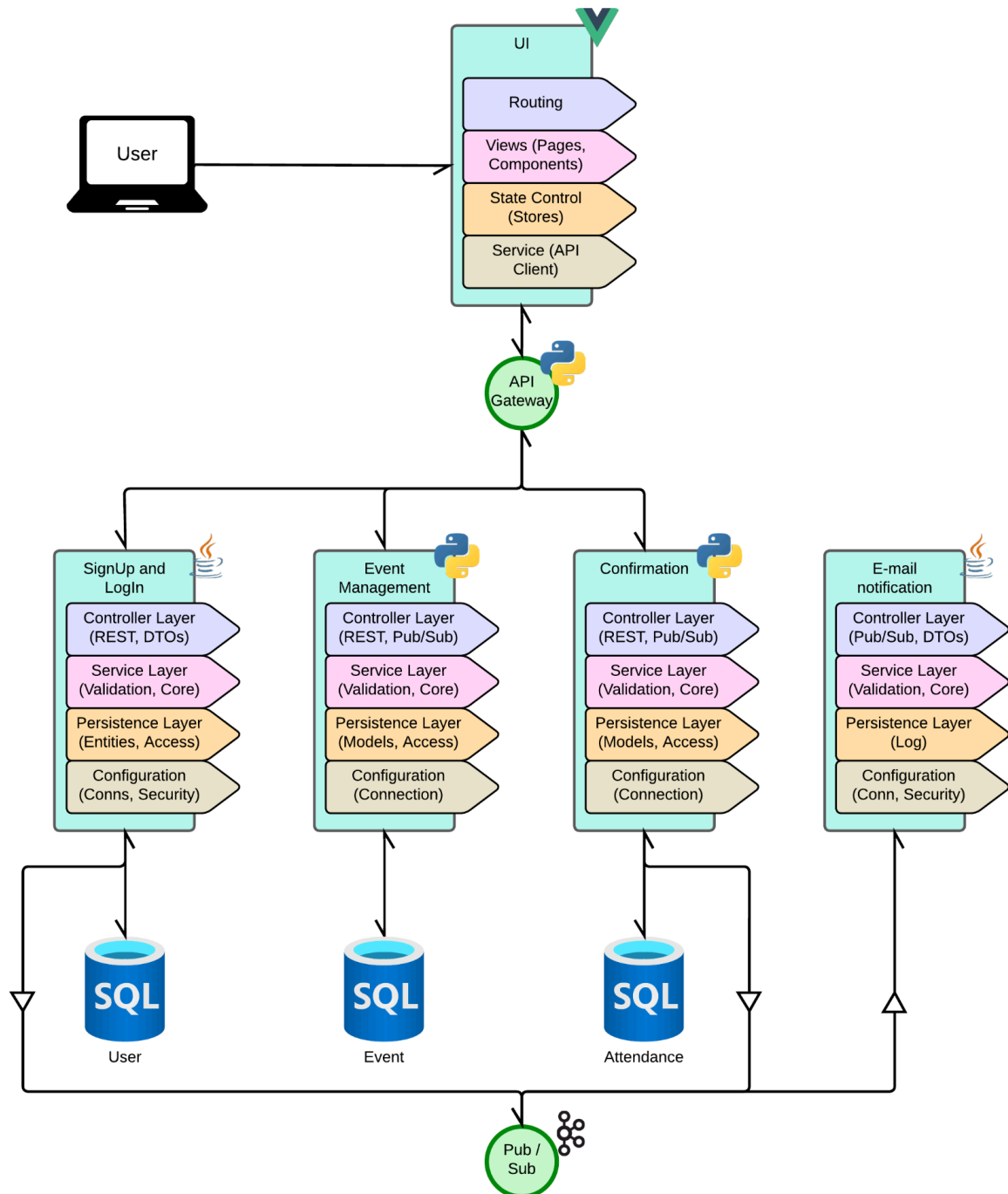
Description: This process depicts how a guest confirms their attendance to an event they have been invited to. It begins when the guest receives a link to a specific event, allowing them to fill an identification form (just asking for personal id) to check if there are existing attendances for that guest in that event.

If no record is found, the system loads an empty form requesting essential information for the attendance; else, the system loads a prefilled form with previously saved information. After that, the guest can then fill in or update the information and submit the form.

The application validates its format and, if there are no format errors, it saves the information in the database. Finally, the system sends a confirmation email to the guest, and updates the organizer's view to reflect the number of confirmed attendees for the event.

Role in the application: This process is one of the most valuable core functionalities of the system. It allows a guest (an external user, without a registered account) to confirm their attendance to an event. This process completes the management basis of event resources, since the information of each guest is sent to the organizer.

4. Architecture Diagram



As shown in the picture above, the project will be developed using a REST API based microservices architecture. It is composed by four main modules:

- **The SignUp and Login Microservice:** Its responsibilities are the storage of all the users personal information when a new account is created, the verification of the credentials for the login process and the update of the password when it is required.
- **The Event Management Microservice:** Its responsibilities are the creation, storage, deletion and update of all the event information associated with a user, and the link generation process to share the event with the guests.
- **The Confirmation Microservice:** Its responsibilities are the processing and storage of the confirmed guests information, and its update when it is required. Additionally, when an event capacity is at its limit, this module manages the consequential waiting list.
- **The E-mail Notification Microservice:** Its responsibilities are the acknowledgment via e-mail of the following successful events:
 - SignUp.
 - Password change.
 - Attendance confirmation.
 - Capacity reached.
 - Waitlist.
 - Waitlist promotion.

Furthermore, the project includes other three relevant pieces:

- **The User Interface:** The main endpoint to connect the user with the application.
- **The API Gateway:** The router for the synchronous REST API communications between the application components.
- **The Pub/Sub:** The event buffer for the asynchronous requests between components, specifically those addressed to the E-Mail Notification Microservice.

a. Further Considerations

The decision to adopt a microservice architecture is driven by the need for modularity, fault tolerance, and operational flexibility. By dividing the system into discrete services, each microservice becomes responsible for a well-defined segment of the domain. This isolation makes it possible to deploy updates incrementally, reducing system-wide downtime and risk. At the same time, developing the project this way provides the developer team a great learning experience.

Microservices also enable horizontal scalability at a granular level. Instead of scaling the entire system uniformly, only the services experiencing higher demand require additional resources.

This efficiency is particularly beneficial in domains with uneven workload distribution, such as event-driven systems where certain operations (for example, sending notifications) may spike significantly more than others.

Furthermore, microservices offer technological freedom. In this project, services are implemented using both Python and Java, allowing each part of the system to leverage the strengths of the chosen language. Python provides rapid development, excellent support for REST APIs, and a flexible ecosystem suitable for business logic and lightweight communication services. Java, on the other hand, offers strong type safety, higher performance for specific workloads, and robust enterprise-level frameworks that facilitate secure authentication and structured application layering. This heterogeneous environment is possible precisely because the services communicate through well-defined interfaces rather than sharing implementation details.

API Gateway and Pub/Sub

The inclusion of an API Gateway serves several essential purposes. It centralizes routing logic so that the client application (in this case, the Vue front-end) interacts with a single unified endpoint instead of communicating directly with each microservice. This not only simplifies the client-side code but also protects internal services from being directly exposed.

The architecture of Fast Event Manager integrates two different communication styles. On one hand, REST is used for synchronous operations that require an immediate response from the frontend and for certain interactions between microservices. On the other hand, a Kafka-based Pub/Sub mechanism is employed for the propagation of domain events related to notifications, confirmations, and state changes within the system. This combination allows the platform to maintain a synchronous interaction model when immediacy is essential, and an asynchronous model when the priority is ensuring message delivery and loose coupling between components.

An extended justification for the Kafka Pub/Sub decision can be read in the **Pub/Sub Messages Documentation**.

Organization

Although the microservices are grouped within two main repositories (python-backend and java-backend) each microservice is implemented as a fully isolated project with its own directory, dependencies, Dockerfile, configuration, and startup process. This organization does not compromise the autonomy of the services. The separation at the code level is preserved by ensuring that no dependencies or shared modules exist between services unless intentionally designed.

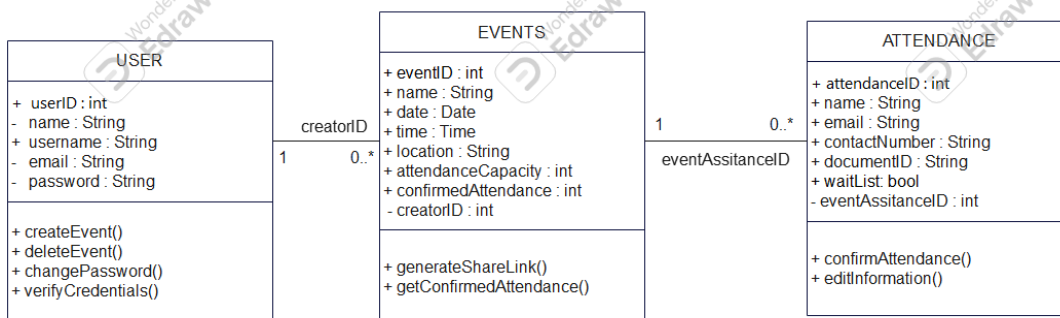
Front-End Considerations

The user interface is implemented using Vue with Vite, which provides a fast, modular, and modern front-end development environment. Vue's component-based architecture aligns with the broader goal of building a system composed of reusable, maintainable, and independently testable modules. The internal structure of the UI mirrors the layered backend architecture; this consistency improves maintainability while promoting clean separation between UI rendering, navigation, state management, and remote communication.

The UI communicates exclusively with the API Gateway, which prevents tight coupling between the front-end and the underlying microservices. The Service layer inside the UI acts as a simple REST client responsible for capturing HTTP logic, leaving the views and components free from networking concerns.

5. Class Diagram

This was the initial idea of the class diagram taking into account the principal CRC cards:



In this web app, **one user can create multiple events**, but each event is created by **only one user**.

Similarly, **one event can have multiple attendances**, and **each attendance record belongs to a single event**.

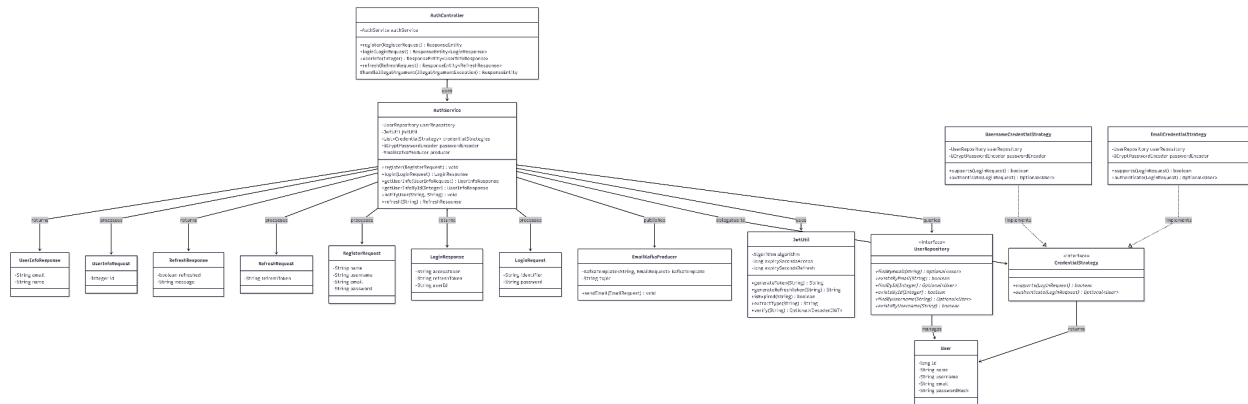
The relationships between the classes are represented as follows:

- The **USER** class is linked to the **EVENTS** class through the attribute **creatorID**, which stores the ID of the user who created the event.
- The **EVENTS** class is linked to the **ATTENDANCE** class through the attribute **eventAssistanceID**, which represents the ID of the event each attendance belongs to.

a. Specific Class Diagrams

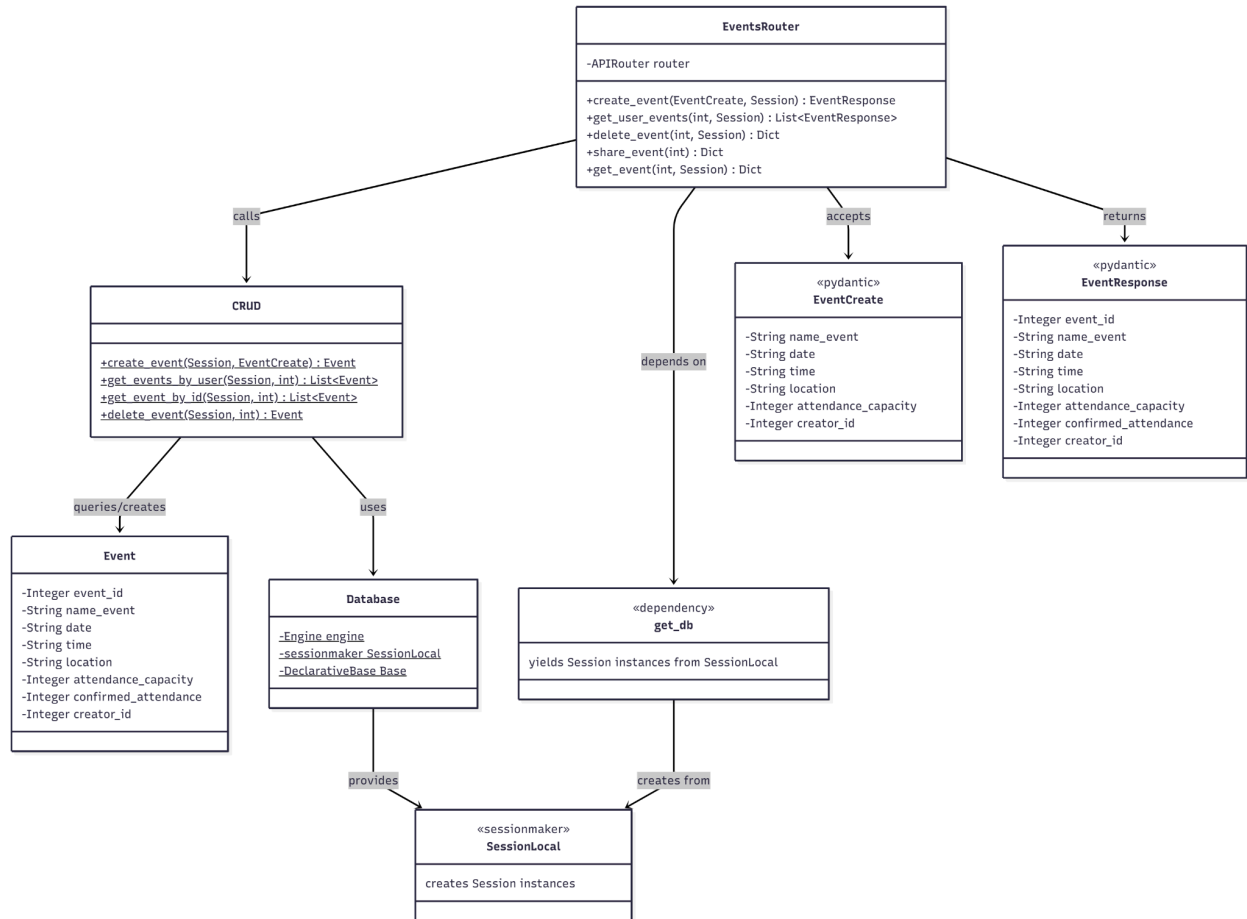
Hence, there are the classes diagrams that were developed for each microservice:

- **SignUp and LogIn:**



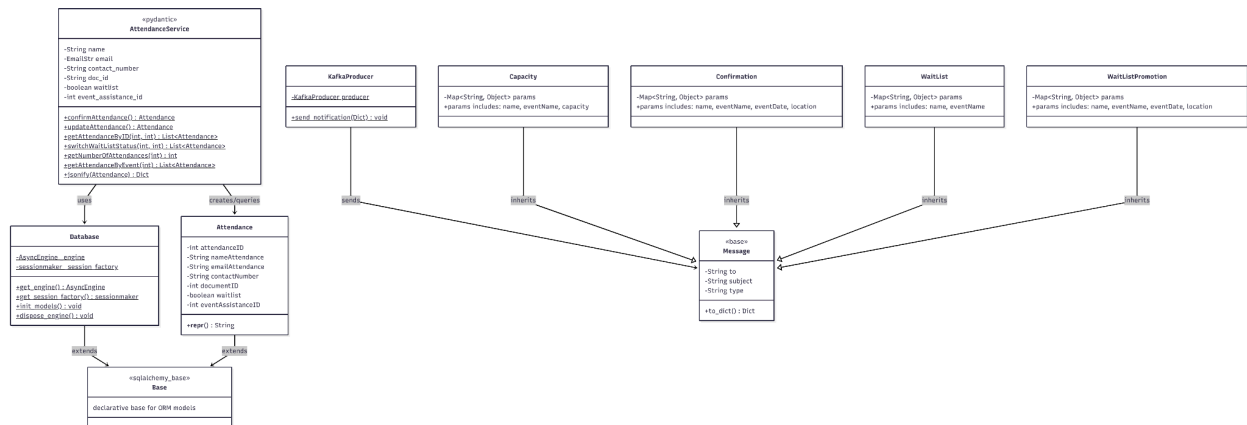
Also available in this link:  [signup-login.png](#)

- **Event Management:**



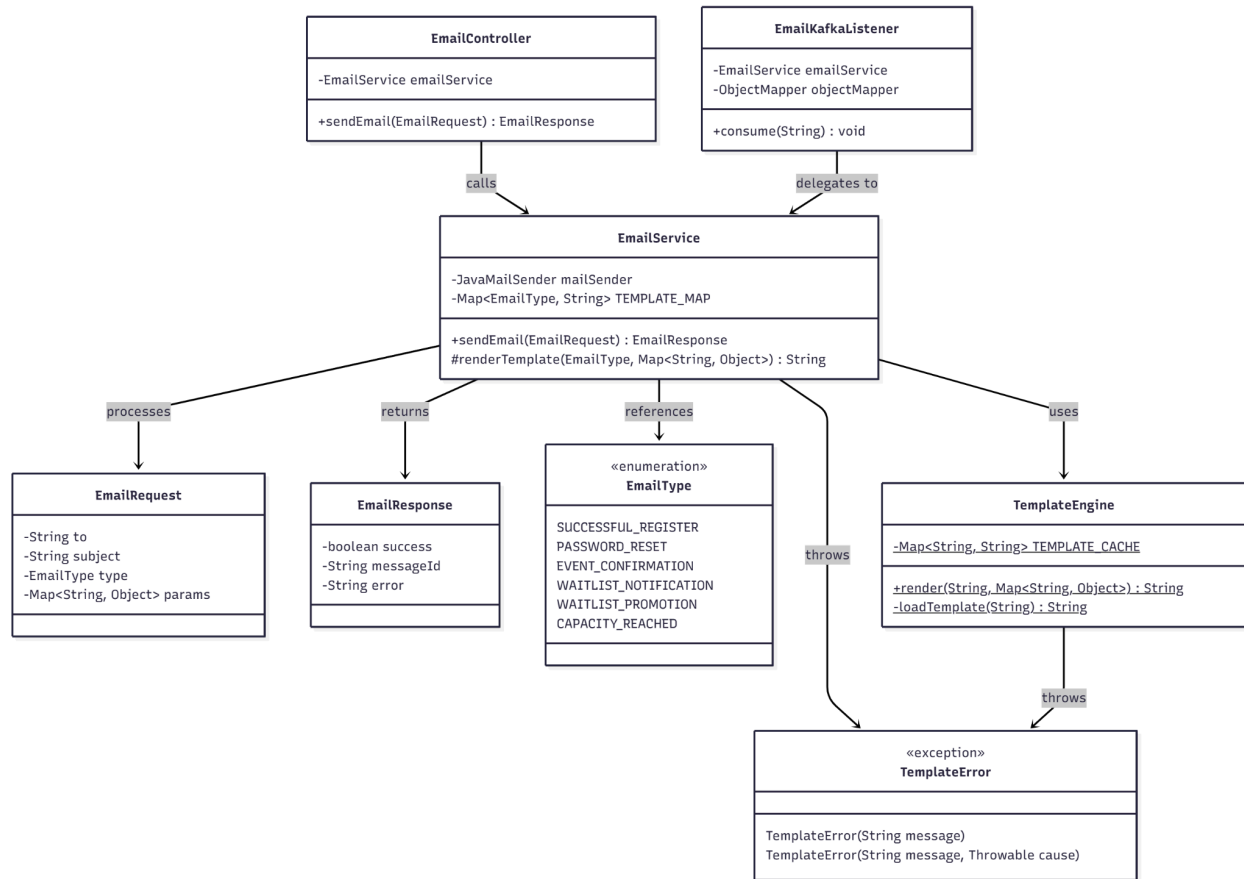
Also available in this link: [📄 event.png](#)

Confirmation:



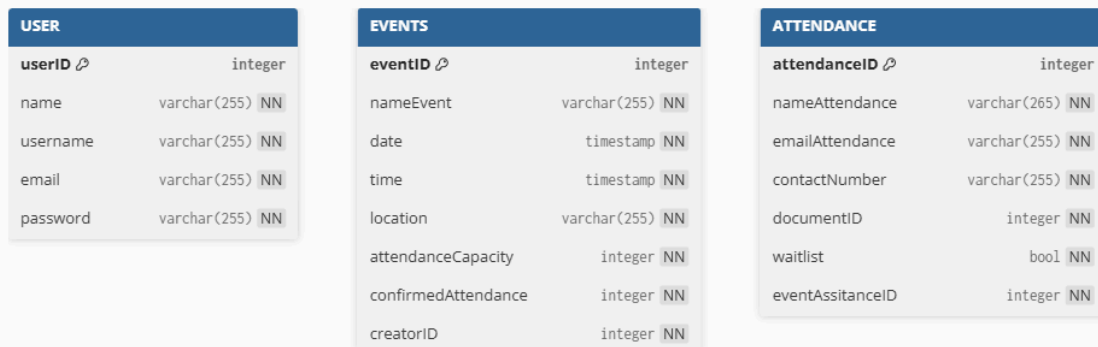
Also available in this link: [📄 confirmation.png](#)

- E-mail Notification:



Also available in this link: [📄 email.png](#)

6. Relational Database Diagram



The relational database diagram for the app is composed of three main tables: **USER**, **EVENTS**, and **ATTENDANCE**, which correspond to the entities defined in the class diagram. Each table has a primary key (PK) and the relationships between tables are established through foreign keys (FK).

Primary Keys (PK)

- **USER.userID** is the primary key used to uniquely identify each user.
- **EVENTS.eventID** is the primary key used to uniquely identify each event.
- **ATTENDANCE.attendanceID** is the primary key used to uniquely identify each attendance record.

Foreign Keys (FK) and Relationships

Even if the relationships and FKs are not shown in the diagram, they will still be implemented through the REST API communication. The project does this abstraction, because each table is actually a detached database of a different microservice; hence, the tables only know each other through their FK simulated attributes:

- The **EVENTS** table contains the “foreign key” **creatorID**, which references **USER.userID**, establishing a **one-to-many relationship** in which one user can create multiple events.
- The **ATTENDANCE** table contains the “foreign key” **eventAssistanceID**, which references **EVENTS.eventID**, establishing a **one-to-many relationship** in which one event can have multiple attendance records.

a. Future Improvement

The current database design intentionally stores attendee information directly within the Attendance table rather than separating it into a dedicated Person entity. This approach is appropriate for the initial phase of the project, as attendees are not required to create user accounts and their interaction with the system is limited to submitting a registration form for a single event. By avoiding an additional table and relationship layer, the design remains simpler, easier to implement, and sufficient for the expected early use cases. Any resulting data redundancy is considered manageable at this stage.

However, as the platform evolves, it is likely that some individuals will register for multiple events, which would lead to repeated personal information across multiple attendance records. To improve data consistency, support richer analytics, and reduce duplication, future iterations of

the system should consider introducing a dedicated **Person** table and using it as a reference in the Attendance table. This enhancement would align the schema with standard normalization practices while maintaining compatibility with the existing functional requirements.